Department of Computer Science and Engineering
Premier University

CSE 482: Contemporary Course of Computer Science & Laboratory

Lab Report 06: Scale and Load Balance Your Architecture

Submitted by:

Name	Mohammad Hafizur Rahman Sakib
ID	0222210005101118
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	04.04.2026

Submitted to:
Avishek Chowdhury
Lecturer, Department of CSE
Premier University,Chattogram

Remarks

Build a Scalable and Highly Available Web Application with Elastic Load Balancing and Auto Scaling

Objective

The main goal of this lab was to make a web application more scalable and fault-tolerant by using Amazon Elastic Load Balancing (ELB) and EC2 Auto Scaling. I created an Amazon Machine Image (AMI) from an existing web server, set up an Application Load Balancer with a target group, built a launch template and an Auto Scaling group, configured target tracking based on CPU utilization, and tested automatic scaling under load.

Key AWS services and concepts covered:

- Amazon EC2 Auto Scaling
- Elastic Load Balancing (Application Load Balancer)
- Target Groups
- Launch Templates
- Amazon Machine Images (AMI)
- CloudWatch Alarms for scaling policies
- High availability across multiple Availability Zones

Scenario

The lab started with a basic infrastructure in the Lab VPC that included a single web server (Web Server 1) running in a public subnet. The goal was to improve this setup by creating a load-balanced and automatically scaling environment. The final architecture used an internet-facing Application Load Balancer in public subnets to distribute traffic, while the Auto Scaling group launched EC2 instances in private subnets across two Availability Zones.

Lab Architecture Overview

The setup included an Application Load Balancer (LabELB) distributing traffic to a target group (LabGroup), which was attached to an Auto Scaling group (Lab Auto Scaling Group) running instances based on the WebServerAMI.

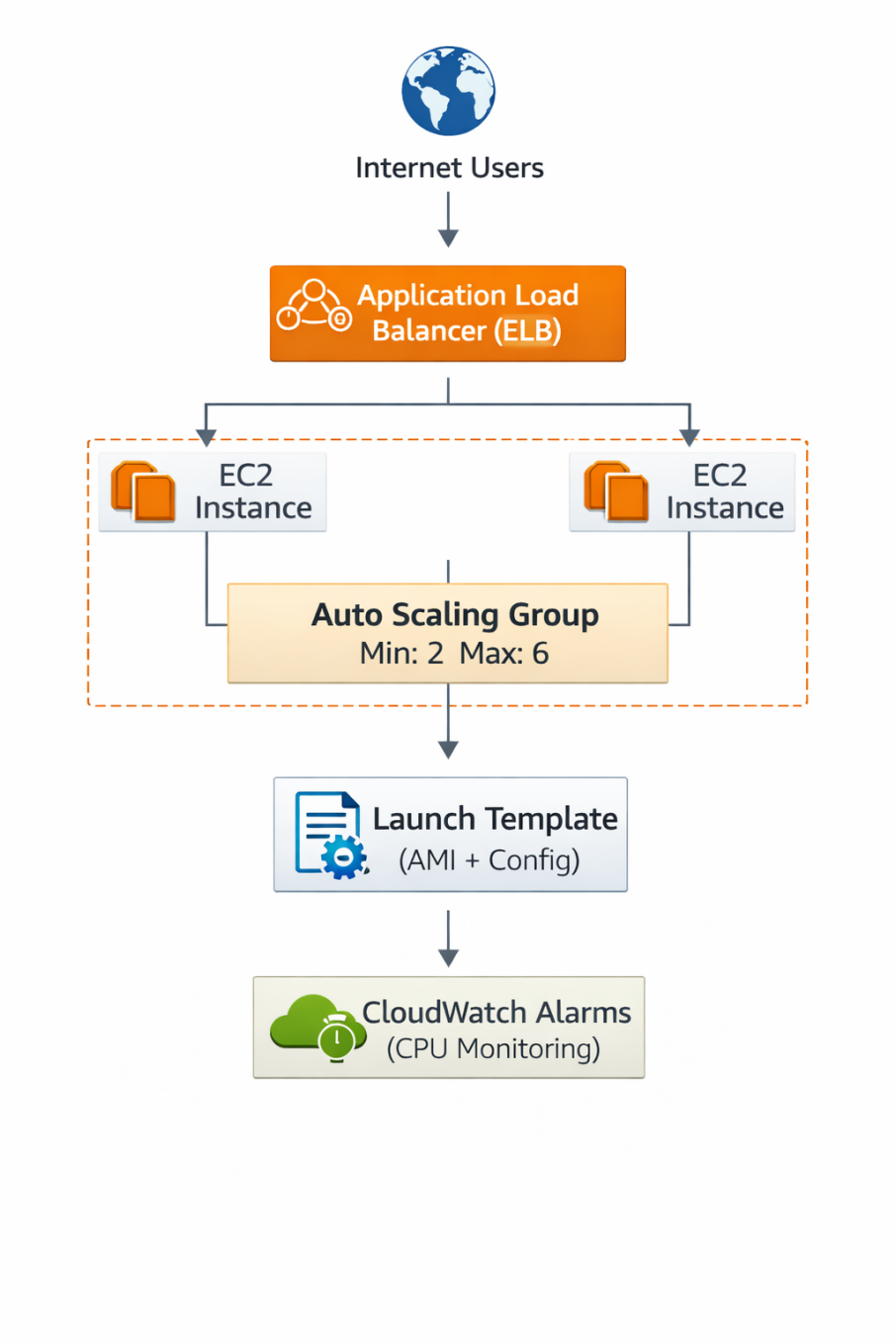

Figure 1: Application Load Balancer distributing traffic to an Auto Scaling group across private subnets in two Availability Zones

This diagram helped visualize how incoming traffic is balanced and how the number of instances automatically adjusts based on demand.

Working Methodology

Task 1: Creating an AMI for Auto Scaling

I went to the EC2 console and selected the running Web Server 1 instance. After confirming its status checks showed 2/2 checks passed, I created an image from it.

- Image name: WebServerAMI
- Image description: Lab AMI for Web Server

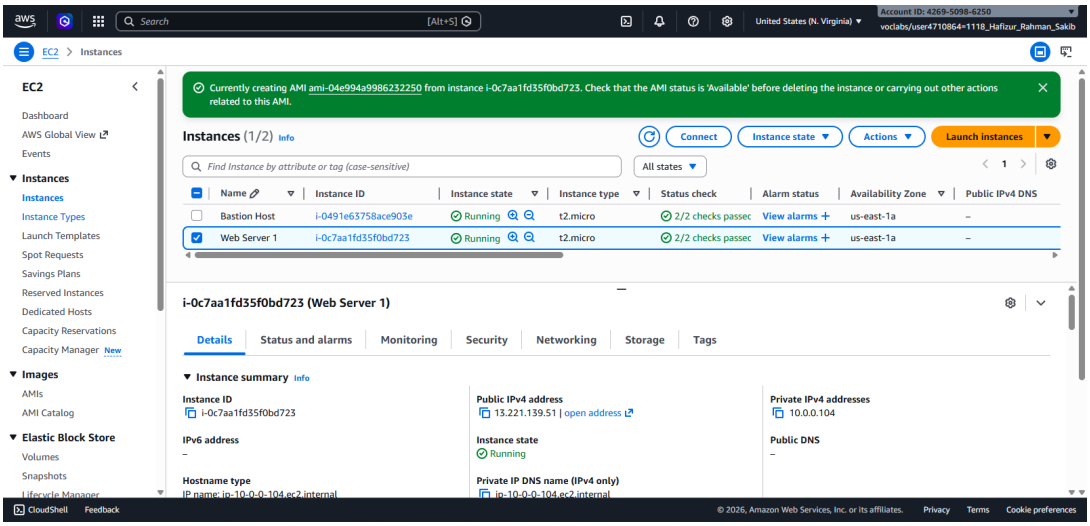

Figure 2: Creating WebServerAMI from the existing Web Server 1 instance

This AMI captured the complete configuration of the web server so that new identical instances could be launched automatically.

Task 2: Creating a Load Balancer

First, I created a target group named LabGroup in the Lab VPC for instances. Then I created an Application Load Balancer named LabELB.

Key settings included:

- VPC: Lab VPC
- Subnets: Public Subnet 1 and Public Subnet 2 (across two Availability Zones)
- Security group: Web Security Group (default security group removed)
- Listener: HTTP on port 80 forwarding to LabGroup

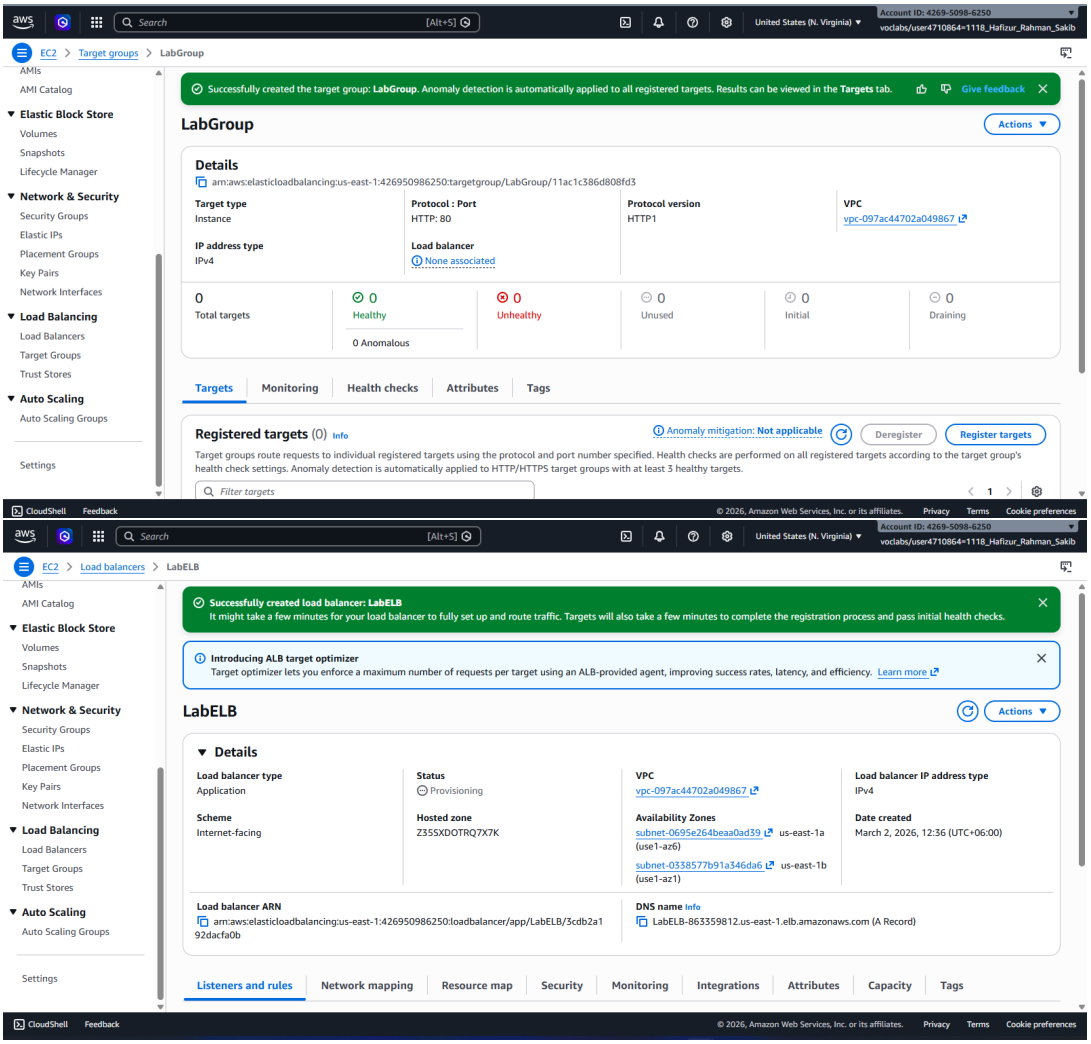

Figure 3: Application Load Balancer LabELB with target group LabGroup

The load balancer was configured to be internet-facing and to distribute traffic evenly across instances in different Availability Zones.

Task 3: Creating a Launch Template and an Auto Scaling Group

I created a launch template named LabConfig using the WebServerAMI.

Key settings in the launch template:

- AMI: WebServerAMI
- Instance type: t2.micro
- Key pair: vockey
- Security group: Web Security Group
- Detailed CloudWatch monitoring: Enabled

Next, I created an Auto Scaling group named Lab Auto Scaling Group using this launch template. The group was placed in Private Subnet 1 and Private Subnet 2, attached to the LabGroup target group, and configured with:

- Desired capacity: 2
- Minimum capacity: 2
- Maximum capacity: 6
- Scaling policy: Target tracking (Average CPU Utilization = 60%)
- Name tag: Lab Instance

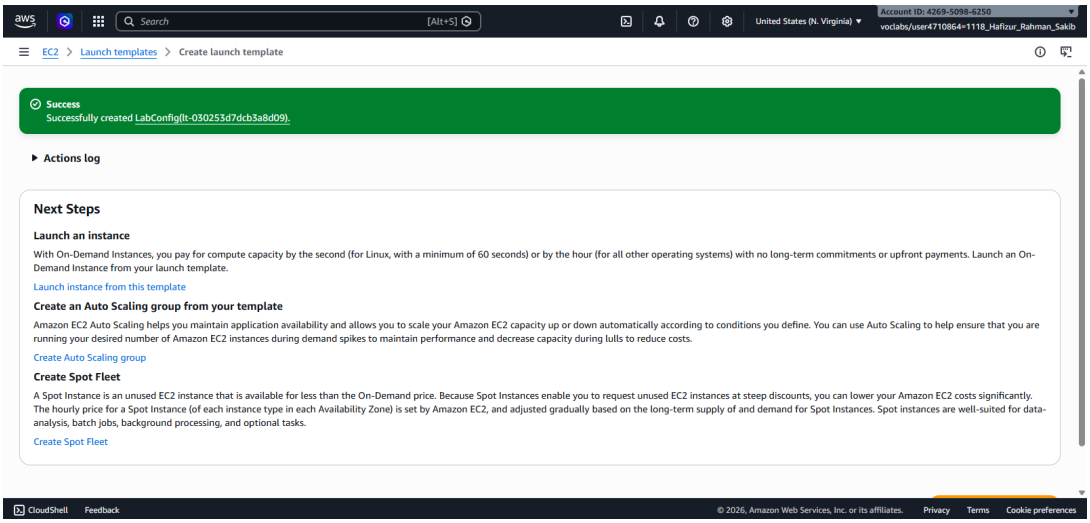

Figure 4: Auto Scaling group Lab Auto Scaling Group with target tracking policy

Auto Scaling automatically launched two instances to meet the desired capacity.

Task 4: Verifying Load Balancing

I checked the EC2 Instances console and confirmed two new instances named Lab Instance were running. In the Target Groups console, both targets in LabGroup showed as healthy.

I copied the DNS name of LabELB and opened it in a browser. The web application loaded successfully, confirming that the load balancer was correctly routing traffic to the instances behind it.

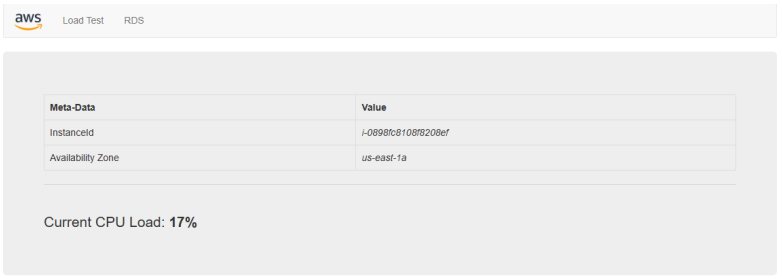

Figure 5: Web application accessed through the LabELB DNS name

Task 5: Testing Auto Scaling

I opened the CloudWatch console and monitored the alarms created by the Auto Scaling group. Then I switched to the web application tab and clicked the Load Test button to generate high CPU load.

Within a few minutes, the AlarmHigh triggered, and Auto Scaling launched additional instances. I refreshed the EC2 Instances view and saw more than two Lab Instance instances running.

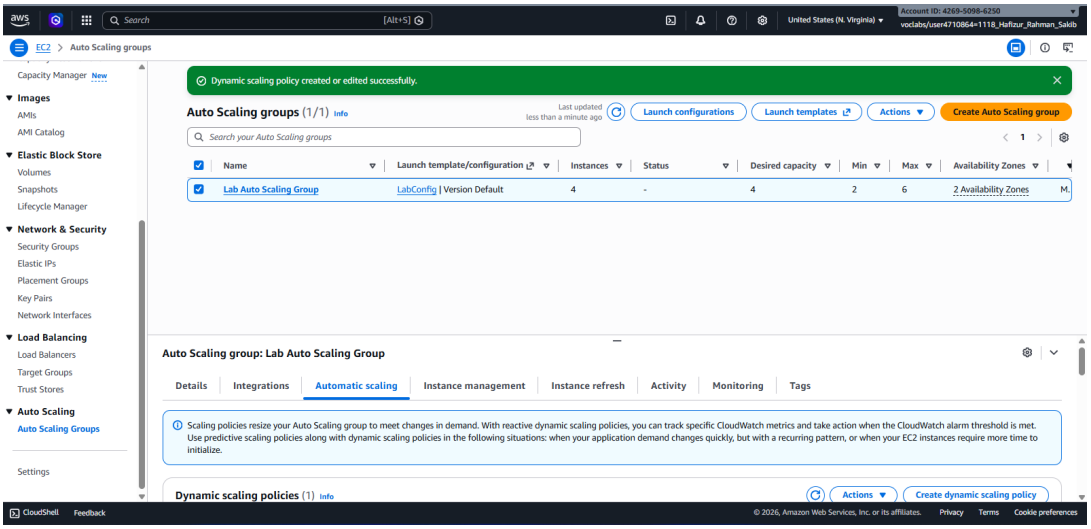

Figure 6: Additional instances launched automatically during the load test

This demonstrated that the target tracking policy successfully scaled the infrastructure based on CPU utilization.

Task 6: Terminating the Original Web Server

Finally, I terminated the original Web Server 1 instance since it was no longer needed (the AMI had already been created and the new instances were handling the workload).

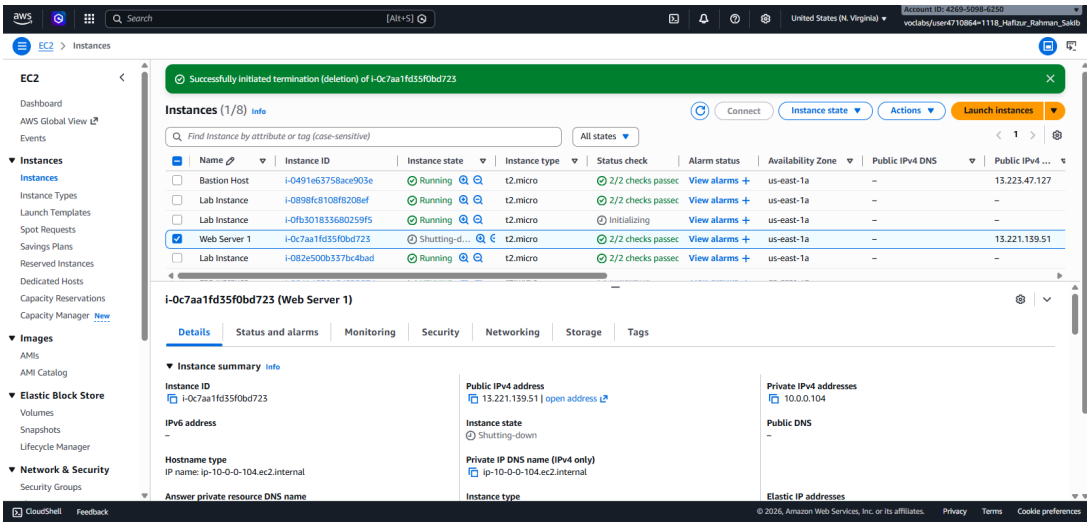

Figure 7: Termination of Web Server

Results Summary

All parts of the lab were completed successfully:

- Created WebServerAMI from the existing web server
- Set up target group LabGroup and Application Load Balancer LabELB
- Created launch template LabConfig and Auto Scaling group Lab Auto Scaling Group
- Configured target tracking scaling policy based on 60% average CPU utilization
- Verified load balancing through the ELB DNS name
- Successfully tested automatic scaling by generating load and observing new instances being launched
- Terminated the original Web Server 1

Conclusion

This lab demonstrated how Elastic Load Balancing and Auto Scaling work together to create a highly available and cost-efficient web application. Instead of managing a single server, the infrastructure can now automatically adjust the number of instances based on demand while the load balancer ensures even traffic distribution and fault tolerance.

I particularly liked how the target tracking policy simplified scaling decisions — I only had to set a target CPU value, and Auto Scaling handled the rest. Using a launch template made it easy to ensure all new instances were identical to the original web server. The ability to place the load balancer in public subnets and the instances in private subnets also improved security.

Overall, this lab helped me understand why these services are essential for real-world applications that experience variable traffic. Managing infrastructure manually would be difficult and error-prone, but with ELB and Auto Scaling, the system becomes resilient and self-managing.